

# AI4GAMMA

## Corso di Formazione su Claude

### *Manuale Utente — Quinta Lezione*

---

Data sessione: 17 aprile 2026

Docente: Stefano Zaghi

Partecipanti: Pierpaolo Laveni, Christian Frassetto, Walter Leggendari, Paolo Bergamaschi

Piattaforma: Microsoft Teams | Durata: ~2,5 ore

## Sommario

|                                                                |    |
|----------------------------------------------------------------|----|
| 1. Introduzione .....                                          | 4  |
| 1.1 Obiettivi della Lezione .....                              | 4  |
| 1.2 Prerequisiti .....                                         | 4  |
| 2. Recap: Metodologia Spec-Driven Development .....            | 5  |
| 2.1 Cos'è lo Spec-Driven Development .....                     | 5  |
| 2.2 I Cinque Principi Cardine .....                            | 5  |
| 2.3 Le Sette Componenti di una Specifica .....                 | 6  |
| 2.4 Il Ciclo Iterativo SDD .....                               | 6  |
| 3. Workflow SDD in Claude Code .....                           | 7  |
| 3.1 Workflow Tradizionale .....                                | 7  |
| 3.2 Workflow Avanzato: Brainstorming & Specification .....     | 7  |
| Fasi del Workflow Avanzato .....                               | 7  |
| 4. Workflow Operativi .....                                    | 8  |
| 4.1 Workflow Lite .....                                        | 8  |
| 4.2 Workflow Intermediate .....                                | 8  |
| 4.3 Workflow Advanced .....                                    | 9  |
| 5. Configurazione Progetto Claude .....                        | 10 |
| 5.1 I Tre Artefatti di Configurazione .....                    | 10 |
| 5.2 Esempio di Prompt per la Configurazione del Progetto ..... | 10 |
| 5.3 Comportamento di Claude .....                              | 11 |
| 6. Claude Cowork .....                                         | 12 |
| 6.1 Differenze tra Chat, Claude Code e Cowork .....            | 12 |
| 6.2 Quando Usare Cowork .....                                  | 12 |
| 6.3 Come Avviare un Task in Cowork dal Progetto .....          | 13 |
| 6.4 La Working Folder di Cowork .....                          | 13 |
| 6.5 Coda di lavoro .....                                       | 13 |
| 7. CLAUDE.md: Il Sistema Operativo del Progetto .....          | 14 |
| 7.1 Cos'è il CLAUDE.md .....                                   | 14 |
| 7.2 Struttura Consigliata del CLAUDE.md .....                  | 14 |
| 7.3 Modularizzazione del CLAUDE.md .....                       | 14 |
| 7.4 Come Generare il CLAUDE.md (Workflow Intermediate) .....   | 15 |
| 7.5 Come Generare il CLAUDE.md (Workflow Advanced) .....       | 15 |
| 8. Workflow Brainstorming & Specification .....                | 16 |
| 8.1 Avviare il Brainstorming .....                             | 16 |
| 8.2 La Fase di Brainstorming .....                             | 16 |
| 8.3 Generazione della Specifica .....                          | 17 |
| 8.4 Generazione del Piano di Sviluppo .....                    | 17 |
| 8.5 Aggiornamento del Piano di Sviluppo Macro .....            | 17 |
| 9. Esecuzione: Sub-Agent Driven Development .....              | 18 |
| 9.1 Avviare lo Sviluppo .....                                  | 18 |

|                                                                         |    |
|-------------------------------------------------------------------------|----|
| 9.2 Esecuzione Sequenziale vs Parallela .....                           | 18 |
| 9.3 Come Funzionano i Sub-Agenti .....                                  | 18 |
| 10. Slash Commands Avanzati .....                                       | 19 |
| 10.1 /revise-claude-md — Aggiornare la Configurazione .....             | 19 |
| 10.2 /claude-md-improver — Migliorare la Configurazione .....           | 19 |
| 10.3 /automation-recommender — Suggerire Automazioni .....              | 20 |
| 10.4 /advisor — Consulente per Situazioni Critiche .....                | 20 |
| 11. Gestione dei Token .....                                            | 21 |
| 11.1 Le Operazioni Più Costose .....                                    | 21 |
| 11.2 Strategie per Ottimizzare il Consumo .....                         | 21 |
| 11.3 Indicazioni per il Piano Gratuito .....                            | 21 |
| 12. Riepilogo: Il Flusso Completo.....                                  | 22 |
| Fase 1 — Ideazione e Analisi (Claude Desktop, Chat normale) .....       | 22 |
| Fase 2 — Configurazione Progetto Claude (Claude Desktop, Progetto)..... | 22 |
| Fase 3 — Generazione della Configurazione (Cowork dal Progetto) .....   | 22 |
| Fase 4 — Sviluppo Iterativo (Claude Code).....                          | 22 |
| 13. Glossario dei Termini Chiave .....                                  | 23 |
| 14. Anticipazioni sulla Prossima Lezione .....                          | 24 |

# 1. Introduzione

---

La sessione ha approfondito in modo pratico la metodologia Spec-Driven Development (SDD), introducendo tre workflow operativi per la configurazione di Claude Code e guidando i partecipanti nella costruzione completa di un progetto prototipale dall'idea iniziale fino all'avvio dello sviluppo.

La lezione ha rappresentato il momento di massima complessità dell'intero percorso formativo, combinando concetti teorici precedentemente introdotti con una dimostrazione pratica passo-passo. L'esercizio fil rouge è stato lo sviluppo di una web app per la gestione delle liste della spesa condivise, denominata Shopping List, ispirata all'applicazione mobile Buy Me a Pie.

## 1.1 Obiettivi della Lezione

- Consolidare i principi teorici della metodologia Spec-Driven Development
- Impostare workflow operativi per la configurazione di Claude Code
- Generare e strutturare il file CLAUDE.md per Claude Code
- Applicare il workflow Brainstorming & Specification
- Comprendere il Sub-Agent Driven Development per lo sviluppo parallelo

## 1.2 Prerequisiti

Per trarre pieno vantaggio da questo manuale, i partecipanti dovrebbero avere familiarità con:

- Utilizzo di base di Claude Desktop (Chat, Progetti, Artefatti)
- Installazione e avvio di Claude Code (terminale o VS Code)
- Concetti fondamentali della metodologia SDD introdotti nella quarta lezione
- Nozioni di base di sviluppo web (non indispensabili per la comprensione teorica)

## 2. Recap: Metodologia Spec-Driven Development

La sessione si è aperta con un riepilogo strutturato dei concetti cardine introdotti nella lezione precedente, fornendo il contesto teorico necessario per l'attività pratica successiva.

### 2.1 Cos'è lo Spec-Driven Development

Lo Spec-Driven Development è una metodologia di sviluppo software in cui l'essere umano guida e focalizza il lavoro dell'intelligenza artificiale anziché lasciarlo libero di procedere autonomamente. Il principio fondamentale è: prima si definisce cosa fare, poi si delega all'AI il come farlo.

 **Principio chiave:** La specifica è il contratto tra noi e l'AI. Più è precisa, più è alta la qualità e la riproducibilità del risultato.

Questa metodologia non è un'invenzione legata all'AI: scrivere le specifiche prima di iniziare qualsiasi sviluppo è una buona pratica consolidata da decenni nel mondo del software. Ciò che cambia con l'AI è la velocità con cui si passa dalla specifica al prototipo funzionante.

 **Distinzione importante:** Lo SDD non è 'vibe coding'. Il vibe coding è un approccio incontrollato che segue l'ispirazione del momento. Lo SDD è un metodo rigoroso e strutturato che richiede più tempo, ma garantisce risultati definiti, mantenibili e riproducibili.

### 2.2 I Cinque Principi Cardine

1. **Analisi preventiva:** Prima di qualsiasi attività, analizzare il progetto e definire chiaramente i requisiti.
2. **Separazione cosa/come:** Le specifiche descrivono il cosa fare; l'AI determina il come farlo (salvo vincoli espliciti).
3. **Decomposizione granulare:** Suddividere il progetto in task atomici, ognuno con il proprio ciclo analisi-sviluppo-review.
4. **Human-in-the-Loop:** L'essere umano deve essere sempre al centro del processo ciclico, controllando ogni step.
5. **Iterazione strutturata:** Il ciclo Specifica → Implementazione → Review → Raffinamento garantisce qualità progressiva.

## 2.3 Le Sette Componenti di una Specifica

Una specifica ben scritta deve includere i seguenti elementi:

| Componente               | Descrizione                                                                                    |
|--------------------------|------------------------------------------------------------------------------------------------|
| Overview                 | Contestualizzazione del problema e definizione dell'obiettivo generale.                        |
| Requisiti Funzionali     | Il cuore della specifica: cosa deve fare il sistema.                                           |
| Requisiti Non Funzionali | Aspetti tecnici, vincoli architetturali e non funzionali (performance, sicurezza, ecc.).       |
| Criteri di Accettazione  | Come verifichiamo di aver ottenuto il risultato desiderato? Cosa rende la specifica testabile? |
| Casi d'Uso               | Come ci aspettiamo di utilizzare la soluzione? Gli use case guidano lo sviluppo.               |
| Gestione degli Errori    | Prevedere fin dall'inizio le casistiche di errore e come gestirle.                             |
| Stack Tecnologico        | Se definito, specificare le tecnologie e i vincoli di implementazione da rispettare.           |

## 2.4 Il Ciclo Iterativo SDD

Il flusso di lavoro si articola in quattro step ciclici:

- **1. Specifica:** Definire in dettaglio cosa vogliamo costruire.
- **2. Piano di Sviluppo:** L'AI elabora un piano dettagliato di come costruirlo.
- **3. Esecuzione:** Sviluppo controllato, task per task.
- **4. Review e Raffinamento:** Verifica del risultato, identificazione di problemi, ritorno alla specifica.

 **Valore strategico:** Questo approccio vale in qualsiasi dominio, non solo nel software. Chiunque voglia ottenere un risultato di qualità — che si tratti di acquistare un macchinario, richiedere una ristrutturazione o sviluppare un'applicazione — deve prima redigere specifiche chiare.

## 3. Workflow SDD in Claude Code

Esistono due modalità principali per implementare la metodologia SDD con Claude Code, a seconda del livello di strutturazione desiderato.

### 3.1 Workflow Tradizionale

Il workflow tradizionale è quello originale, adottabile anche senza strumenti specializzati.

Si basa su:

- **Prompt strutturato:** La nostra richiesta, eventualmente corredata da documenti di specifica esterni. Il prompt strutturato segue lo schema TCOF (Task, Context, Output, Format).
- **Planning Mode:** Si attiva Claude Code in una modalità di sola lettura in cui lo strumento ragiona su ciò che deve fare senza modificare alcun file. Questo produce il piano di sviluppo dettagliato.
- **Specifica = Prompt:** In questo workflow, la specifica coincide con il prompt iniziale e i documenti allegati.
- **Esecuzione controllata:** Sviluppo task per task con review intermedia.

 **Attivazione Planning Mode:** Il planning mode si attiva tramite il menu a tendina in Claude Code oppure includendo nel prompt il trigger esplicito: 'planning mode required'.

### 3.2 Workflow Avanzato: Brainstorming & Specification

Il workflow avanzato, denominato Brainstorming & Specification, è la trasposizione potenziata del flusso tradizionale. Prevede un flusso di lavoro guidato attivato tramite slash command disponibili tramite plugin installati in Claude Code.

#### Fasi del Workflow Avanzato

1. **Brainstorming collaborativo:** Con il comando /brainstorming si avvia una sessione interattiva in cui l'AI pone domande mirate per chiarire e approfondire le specifiche. Il dialogo maieutico aiuta a scoprire aspetti non considerati inizialmente.
2. **Specifica dettagliata:** Al termine del brainstorming, l'AI genera un documento di specifica strutturato, più ricco e completo di quanto si otterrebbe con il workflow tradizionale.
3. **Piano di sviluppo:** Dalla specifica approvata viene generato automaticamente il piano di sviluppo (sempre in planning mode).
4. **Esecuzione (Sub-Agent Driven):** La fase di sviluppo può essere eseguita sequenzialmente o in parallelo delegando task a sub-agenti.

 **Raccomandazione:** Il workflow avanzato è sempre preferibile quando si ha tempo sufficiente. Produce specifiche di qualità superiore e piani di sviluppo più strutturati, con conseguente miglioramento della qualità del codice generato.

## 4. Workflow Operativi

Durante la lezione sono stati presentati tre workflow dedicati alla configurazione dell'ambiente di sviluppo in Claude Code, calibrati in base alla complessità del progetto e al livello di strutturazione desiderato.

### 4.1 Workflow Lite

Il workflow più rapido, adatto a prototipi semplici o a chi si avvicina per la prima volta al metodo.

- **Fase 1:** Definire l'idea dell'applicazione e chiedere all'AI suggerimenti sulle funzionalità possibili.
- **Fase 2:** Selezionare le funzionalità desiderate e chiedere un'analisi dello stack tecnologico.
- **Fase 3:** Richiedere la generazione del file CLAUDE.md (configurazione di Claude Code).
- **Fase 4:** Avviare il ciclo SDD in Claude Code con il workflow Brainstorming & Specification.

**Limite:** La qualità della specifica e della configurazione dipende direttamente dall'accuratezza del prompt iniziale. Senza una struttura di progetto a supporto, il risultato è meno prevedibile.

### 4.2 Workflow Intermediate

Il workflow raccomandato per la maggior parte dei casi d'uso. Introduce l'uso dei Progetti Claude Desktop come base strutturata.

1. **Brainstorming iniziale in Claude Desktop:** Partire dall'idea e interagire con Claude per esplorare funzionalità, possibili approcci e stack tecnologici.
2. **Creazione di un Progetto Claude:** Usare la tecnica del metaprompting per generare i tre artefatti di configurazione del progetto: descrizione sintetica, descrizione dettagliata (project context) e istruzioni di progetto (system prompt dell'agente).
3. **Arricchimento della knowledge base:** Eseguire analisi specifiche in Claude Cowork (es. analisi funzionale dettagliata, analisi dello stack tecnologico, SRS) e caricare i documenti risultanti nel progetto.
4. **Generazione del CLAUDE.md:** All'interno del progetto, richiedere la generazione del file di configurazione per Claude Code, ottimizzato e modularizzato.
5. **Sviluppo in Claude Code:** Usando il CLAUDE.md generato come punto di partenza, avviare il workflow SDD per lo sviluppo del prototipo.

💡 **Vantaggio chiave:** Lavorare all'interno di un Progetto Claude garantisce che ogni interazione abbia accesso all'intera knowledge base accumulata, producendo output molto più contestualizzati e coerenti.

## 4.3 Workflow Advanced

Estende il workflow Intermediate aggiungendo un set di documenti di configurazione avanzata come input per la generazione del CLAUDE.md. Si tratta di cinque guide specializzate:

- **1. Best Practice Claude Code Config:** Linee guida per la configurazione ottimale di Claude Code.
- **2. Enforcement Rules:** Regole proattive che ricordano a Claude di rispettare i principi cardine della programmazione (file modulari, funzioni brevi, best practice).
- **3. Modularization Best Practice:** Indicazioni specifiche su come modularizzare correttamente i file di configurazione.
- **4. File Referencing Guide:** Guida per referenziare correttamente i file nel CLAUDE.md (caricamento lazy vs eager).
- **5. Universal Cross-Framework Best Practice:** Best practice di sviluppo software applicabili a qualsiasi framework.

Con questi cinque documenti nel progetto, la generazione del CLAUDE.md produce una configurazione di qualità nettamente superiore, perché Claude ha in 'pancia' tutte le conoscenze necessarie per prendere le decisioni migliori.

 **Nota temporale:** Il workflow Advanced richiede più tempo nella fase di setup iniziale. È la scelta ottimale per progetti strutturati e di lungo periodo, meno indicata per prototipi rapidi.

## 5. Configurazione Progetto Claude

La fase di configurazione del progetto costruisce, di fatto, un agente AI specializzato. Il progetto diventa la base di conoscenza condivisa tra Claude Desktop e Claude Code.

### 5.1 I Tre Artefatti di Configurazione

La prima azione da compiere è generare, tramite metaprompting, i tre documenti che costituiscono la configurazione di base del progetto:

- **Descrizione Sintetica:** Breve sommario del progetto (2-4 righe). Viene inserita come titolo/descrizione del progetto in Claude Desktop.
- **Descrizione Dettagliata (Project Context):** Documento esteso che descrive il progetto in modo completo: obiettivi, funzionalità, contesto di utilizzo. Viene caricato come file nella knowledge base del progetto.
- **Istruzioni di Progetto:** Il system prompt dell'agente che stai costruendo. Definisce il comportamento, il tono, le priorità e i vincoli operativi di Claude nel contesto del progetto. Viene inserito nel campo 'Istruzioni' del progetto.

 **Metaprompting:** La tecnica del metaprompting consiste nel chiedere all'AI di scrivere il miglior prompt per eseguire un compito, anziché eseguirlo direttamente. Questo produce prompt di qualità superiore, ottimizzati per il contesto specifico.

### 5.2 Esempio di Prompt per la Configurazione del Progetto

Esempio di prompt utilizzato nella sessione per generare i tre artefatti via metaprompting:

```
"Scrivi il miglior prompt per Claude per richiedere la generazione di tre contenuti sotto forma di artefatti scaricabili: (1) una descrizione sintetica del progetto Shopping List; (2) una descrizione dettagliata ed esaustiva; (3) le istruzioni di progetto per Claude. Generami questo prompt rispettando i seguenti requisiti aggiuntivi: [requisiti specifici...]"
```

## 5.3 Comportamento di Claude

Quando si lavora all'interno di un progetto, Claude non riceve l'intera knowledge base ad ogni query. Utilizza un meccanismo di retrieval semantico che recupera i chunk (frammenti) più rilevanti rispetto alla richiesta corrente.

Per garantire che legga tutto il contesto in modo completo, qualora necessario, è necessario richiederlo esplicitamente nel prompt, come nell'esempio:

```
"Voglio che leggi e comprendi tutta la documentazione di progetto caricata. Prima di procedere con la fase successiva, assicurati di aver letto e compreso tutti i documenti."
```

**⚠ Attenzione ai token:** Chiedere la lettura completa di tutti i documenti consuma significativamente più token rispetto al retrieval selettivo. Usare questa modalità solo quando il contesto completo è strettamente necessario.

## 6. Claude Cowork

Claude Cowork è uno strumento autonomo della piattaforma Claude, distinto dalla chat normale e da Claude Code. Condivide con Claude Code la logica agentica, ma è progettato per attività di analisi e orchestrazione multitask, non per lo sviluppo di codice.

### 6.1 Differenze tra Chat, Claude Code e Cowork

| Caratteristica       | Chat Normale                                | Claude Cowork                            | Claude Code                            |
|----------------------|---------------------------------------------|------------------------------------------|----------------------------------------|
| Uso primario         | Domande e risposte, testi, analisi semplici | Task complessi, analisi multi-step, file | Sviluppo codice e automazione          |
| Modalità operativa   | Conversazionale                             | Agentica (stessa di Claude Code)         | Agentica con file system               |
| Output tipici        | Testo, tabelle, artefatti                   | Documenti Word, Excel, PPT, artefatti    | File di codice, script, configurazioni |
| Gestione cartelle    | No                                          | Sì (crea working folder locale)          | Sì (lavora su file system)             |
| Adatto a non tecnici | Sì                                          | Sì                                       | Parzialmente                           |

### 6.2 Quando Usare Cowork

Cowork è la scelta ottimale quando:

- Si devono eseguire più task in sequenza con verifiche intermedie.
- L'orchestrazione è complessa e richiede una gestione strutturata dei passi.
- Il risultato atteso è un file (documento Word, Excel, PDF, ecc.) piuttosto che una risposta testuale.
- Si parte da un Progetto Claude e si vuole eseguire un task sfruttando tutta la knowledge base del progetto.

 **Regola pratica:** Chat normale per risposte rapide e one-shot. Cowork per lavori complessi, multi-step, che producono file. Claude Code per tutto ciò che riguarda lo sviluppo software.

## 6.3 Come Avviare un Task in Cowork dal Progetto

Il pulsante "Start a task in Cowork" presente in ogni Progetto Claude avvia automaticamente una sessione Cowork con il progetto come contesto di lavoro. Questo è il modo più efficace per:

- Eseguire analisi complesse utilizzando tutti i documenti del progetto.
- Generare artefatti (documenti, report) coerenti con la knowledge base del progetto.
- Orchestrare flussi di lavoro multi-step con checkpoint intermedi.

## 6.4 La Working Folder di Cowork

Quando Cowork avvia una sessione, crea automaticamente una cartella di lavoro locale (working folder) nella home directory dell'utente. In questa cartella:

- Vengono copiati i documenti del progetto Claude (se si parte da un progetto).
- Vengono salvati tutti gli output generati durante la sessione.
- L'agente lavora e scrive file localmente.

È possibile anche specificare una cartella di lavoro manualmente, senza partire da un progetto, navigando nella sezione dedicata di Cowork e selezionando la directory desiderata.

## 6.5 Coda di lavoro

Cowork gestisce i messaggi dell'utente come una coda di lavoro. Se si invia un messaggio durante l'esecuzione di un task:

- Il messaggio viene accodato.
- Viene elaborato al momento più appropriato nel flusso di lavoro corrente.
- Se riguarda un'attività già completata, l'agente torna indietro, verifica e aggiorna gli output successivi se necessario.

 **Intervento in corso d'opera:** Durante l'esecuzione di un task complesso in Cowork, è possibile inviare domande o richieste di modifica senza interrompere il processo. L'agente le integrerà nel flusso di lavoro nel momento più opportuno.

## 7. CLAUDE.md: Il Sistema Operativo del Progetto

Il file CLAUDE.md è il cuore della configurazione di Claude Code. Viene letto all'avvio di ogni sessione di lavoro e definisce il contesto operativo per tutti gli agenti coinvolti nello sviluppo. È il ponte tra il Progetto Claude Desktop e l'ambiente di sviluppo Claude Code.

### 7.1 Cos'è il CLAUDE.md

Il CLAUDE.md è analogo a un sistema operativo per la cartella di sviluppo: senza di esso, Claude Code non ha una base di conoscenza condivisa. Con un buon CLAUDE.md:

- Ogni agente istanziato parte con lo stesso contesto.
- Le regole di sviluppo sono sempre presenti e rispettate.
- La qualità del codice generato è più alta e consistente.
- Le sessioni di sviluppo successive mantengono la coerenza con le sessioni precedenti.

### 7.2 Struttura Consigliata del CLAUDE.md

Un CLAUDE.md ben strutturato dovrebbe contenere:

- **Scopo e Contesto:** Descrizione breve del progetto e del suo obiettivo principale.
- **Principi Assoluti:** Regole che non devono mai essere violate (es. 'l'app deve sempre funzionare offline', 'massimo 200 righe di codice per file').
- **Stack Tecnologico:** Tecnologie, framework e dipendenze utilizzate nel progetto.
- **Architettura:** Struttura logica dell'applicazione, suddivisione in moduli.
- **Istruzioni Proattive:** Comandi che Claude deve eseguire autonomamente (es. 'dopo ogni task, aggiorna la mappa del progetto', 'monitora il numero di righe di codice').
- **Regole di Lettura File:** Quando leggere quali file (lazy loading vs eager loading).
- **Enforcement Rules:** Promemoria sulle best practice di programmazione da rispettare sempre.

### 7.3 Modularizzazione del CLAUDE.md

Per progetti di medie-grandi dimensioni, è fortemente consigliato suddividere il CLAUDE.md in più file modulari, referenziati dal file principale. I vantaggi sono:

- Riduzione del consumo di token: ogni agente carica solo i file rilevanti per il suo task.
- Manutenzione semplificata: si aggiorna un file piccolo anziché un documento monolitico.
- Chiarezza strutturale: ogni file ha un tema specifico e ben definito.

 **Regola pratica:** Un buon CLAUDE.md modulare prevede file tematici: uno per lo stack, uno per l'architettura, uno per le regole di sincronizzazione, uno per le enforcement rules, ecc. Il file principale fa da indice e dice a Claude quando leggere quale file.

## 7.4 Come Generare il CLAUDE.md (Workflow Intermediate)

Il modo più diretto per generare il CLAUDE.md nel workflow intermediate è tramite un prompt eseguito all'interno del Progetto Claude Desktop:

```
Genera il miglior file CLAUDE.md per lo sviluppo di un MVP della soluzione. Stack tecnologico selezionato: [stack]. Il file di configurazione deve essere ottimizzato per massimizzare la qualità del codice e ridurre al minimo l'utilizzo di token. In particolare, il file CLAUDE.md deve essere modularizzato in file più piccoli con riferimenti tra loro. Tutti i file referenziati devono essere generati in lingua italiana sotto forma di artefatti scaricabili.
```

## 7.5 Come Generare il CLAUDE.md (Workflow Advanced)

Nel workflow advanced, prima di richiedere la generazione del CLAUDE.md si aggiungono al progetto i cinque documenti di best practice avanzate. Poi si usa il metaprompting per generare il prompt ottimale e si esegue il task in Cowork:

1. Caricare nel progetto i 5 file di best practice avanzate.
2. Usare il metaprompting per generare il prompt ottimale per la generazione del CLAUDE.md.
3. Eseguire il prompt in Cowork (pulsante 'Start a task in Cowork' dal progetto).
4. Cowork genera il CLAUDE.md e tutti i file correlati nella working folder locale.
5. Copiare il contenuto della working folder nella cartella di sviluppo.

☑ **Best Practice:** Il CLAUDE.md generato con il workflow advanced produce automaticamente anche un Piano di Sviluppo macro (con sprint) e una Mappa di Progetto di tutti i file. Questi artefatti sono fondamentali per partire con il ciclo SDD in Claude Code.

## 8. Workflow Brainstorming & Specification

Una volta configurato l'ambiente di sviluppo con il CLAUDE.md, si avvia il ciclo SDD in Claude Code. Il punto di ingresso è sempre il workflow Brainstorming & Specification, che garantisce la massima qualità della specifica per ogni singolo task.

### 8.1 Avviare il Brainstorming

Il comando `/brainstorming`, disponibile tramite il plugin installato in Claude Code, avvia una sessione collaborativa interattiva. Il modo corretto di usarlo è:

```
/brainstorming [PROMPT STRUTTURATO] # Task: Sprint Zero - Setup Infrastruttura ##  
Obiettivo Configurare l'architettura base del progetto Shopping List ## Requisiti  
Test Hello World funzionante al termine ## Contesto Leggi il piano di sviluppo in  
/docs/development-plan.md
```

 **Suggerimento:** Più è strutturato il prompt iniziale, più sarà efficiente la fase di brainstorming. Un prompt TCOF ben formulato, unito al riferimento al piano di sviluppo, permette all'AI di partire già con un contesto chiaro e porre domande più mirate.

### 8.2 La Fase di Brainstorming

Durante il brainstorming, Claude pone una serie di domande per chiarire aspetti tecnici e funzionali non specificati. Questo dialogo maieutico serve a:

- Identificare vincoli bloccanti non considerati inizialmente (es. accesso a servizi esterni non disponibili).
- Chiarire decisioni architettureali che influenzeranno l'implementazione.
- Raccogliere preferenze dell'utente su approcci alternativi.
- Definire l'ambito esatto del task (cosa è in scope e cosa è out of scope).

## 8.3 Generazione della Specifica

Al termine del brainstorming, Claude genera automaticamente un documento di specifica formale per il task corrente. Questo documento include:

- Obiettivo e scope del task.
- Decisioni architetturali prese durante il brainstorming.
- Elenco delle dipendenze e dei pacchetti da installare.
- Task rimandati a sprint successivi (out of scope corrente).
- Criteri di accettazione per la verifica del risultato.

La specifica viene salvata nella cartella /specs del progetto. È fondamentale revisionarla prima di procedere.

## 8.4 Generazione del Piano di Sviluppo

Dopo l'approvazione della specifica, Claude genera il piano di sviluppo dettagliato per il task, salvato nella cartella /plans. Questo piano contiene:

- Sequenza dettagliata di azioni da eseguire.
- Per ogni azione: obiettivo, file da creare/modificare, risultato atteso.
- Sezione 'For Agentic Workers': istruzioni per l'esecuzione sequenziale o parallela tramite sub-agenti.

☒ **Best Practice:** Non saltare mai la fase di revisione della specifica prima di approvare la generazione del piano. Una specifica incompleta o errata si traduce inevitabilmente in codice di bassa qualità.

## 8.5 Aggiornamento del Piano di Sviluppo Macro

Al termine di ogni task, è consigliabile richiedere esplicitamente l'aggiornamento del piano di sviluppo macro (il documento di alto livello con tutti gli sprint):

```
"Aggiorna il piano di sviluppo macro in /docs/development-plan.md in base a ciò che abbiamo appreso e deciso durante questo sprint."
```

## 9. Esecuzione: Sub-Agent Driven Development

Dopo aver generato specifica e piano, si avvia la fase di sviluppo. Claude Code offre due modalità di esecuzione, entrambe disponibili tramite la skill 'Super Powers'.

### 9.1 Avviare lo Sviluppo

Il prompt di avvio dello sviluppo include tipicamente:

1. Un riferimento al CLAUDE.md e al piano di sviluppo da seguire.
2. Il trigger per attivare il planning mode (facoltativo, per una review pre-sviluppo).
3. L'istruzione per utilizzare la modalità Sub-Agent Driven per l'esecuzione.

```
# Sprint Zero - Setup Infrastruttura ## Obiettivo [obiettivo dello sprint] ##
Requisiti [requisiti specifici] ## Contesto Leggi il piano di sviluppo macro e la
specifica in /specs/sprint-zero.md e il piano in /plans/sprint-zero-plan.md prima
di iniziare. Planning mode richiesto. Utilizza la skill Super Powers - Sub Agent
Driven Development per implementare questo piano.
```

### 9.2 Esecuzione Sequenziale vs Parallela

| Aspetto           | Esecuzione Sequenziale | Sub-Agent Driven (Parallela)          |
|-------------------|------------------------|---------------------------------------|
| Velocità          | Più lenta              | Più rapida                            |
| Consumo token     | Minore                 | Maggiore (sub-agenti in parallelo)    |
| Qualità output    | Buona                  | Superiore (review automatica inclusa) |
| Review intermedia | Manuale ad ogni step   | Automatica + manuale finale           |

### 9.3 Come Funzionano i Sub-Agenti

Nella modalità Sub-Agent Driven Development, l'agente orchestratore crea istanze figlio (sub-agenti) a cui delega task specifici. Il meccanismo funziona così:

1. L'agente orchestratore analizza il piano e identifica i task parallelizzabili.
2. Per ogni task, istanzia un sub-agente con: il CLAUDE.md come contesto base, le istruzioni specifiche per il task delegato.
3. I sub-agenti lavorano in parallelo senza comunicare tra loro durante l'esecuzione.
4. Al termine, ogni sub-agente invia un report all'orchestratore con i risultati.
5. L'orchestratore consolida i risultati, gestisce eventuali conflitti e coordina le review successive.

 **Review automatica inclusa:** Il piano generato da Claude include già task di code review automatica tra i sub-agenti. Questo non sostituisce la review manuale dell'utente, ma aumenta significativamente la qualità del codice prodotto.

## 10. Slash Commands Avanzati

Il plugin installato in Claude Code mette a disposizione una serie di comandi slash per mantenere aggiornata la configurazione del progetto nel tempo. Questi comandi vanno eseguiti al termine di ogni sessione di sviluppo significativa.

### 10.1 /revise-claude-md — Aggiornare la Configurazione

Questo comando aggiorna il file CLAUDE.md (e tutti i file correlati) integrando le nuove conoscenze e decisioni acquisite durante la sessione di sviluppo corrente.

- Quando usarlo: Al termine di ogni sprint o sessione di sviluppo.
- Come funziona: Claude analizza ciò che ha appreso durante la sessione (nuove soluzioni, problemi incontrati, decisioni architetturali) e propone aggiornamenti al CLAUDE.md.
- Interazione: Non aggiorna in automatico. Presenta le modifiche proposte e chiede conferma prima di applicarle. L'utente può selezionare solo le modifiche rilevanti.

☑ **Best Practice:** Usare /revise-claude-md al termine di ogni sprint mantiene la configurazione allineata con l'evoluzione del progetto, garantendo che le sessioni successive partano con un contesto sempre aggiornato.

### 10.2 /claude-md-improver — Migliorare la Configurazione

Questo comando analizza l'intera code base del progetto e suggerisce come migliorare il file CLAUDE.md per renderlo più efficace.

- Differenza da /revise-claude-md: Non si limita ad aggiungere ciò che si è imparato, ma propone miglioramenti strutturali e ottimizzazioni.
- Output tipico: Una valutazione della qualità del CLAUDE.md corrente (es. '78/100 - livello B') con un elenco di miglioramenti specifici da applicare.
- Interazione: Come il precedente, propone modifiche e chiede approvazione prima di applicarle.

⚠ **Nota critica:** Non tutti i miglioramenti suggeriti sono necessariamente ottimali per il vostro caso specifico. Valutare con attenzione ogni proposta prima di accettarla.

### 10.3 /automation-recommender — Suggerire Automazioni

Questo comando analizza la codebase e il contesto del progetto per suggerire automazioni, plugin, configurazioni MCP o skill aggiuntivi che potrebbero essere utili per lo sviluppo.

- Quando usarlo: Non a ogni sprint, ma quando si è raggiunto un punto avanzato dello sviluppo o si vuole esplorare nuove possibilità di automazione.
- Esempi di suggerimenti: Nuovi plugin da installare, skill specifici per il framework usato, MCP server da collegare, righe di configurazione aggiuntive per il CLAUDE.md.

### 10.4 /advisor — Consulente per Situazioni Critiche

Un comando recente (introdotto nelle ultime settimane) che configura Claude Code per consultare automaticamente un modello più potente quando si trova in difficoltà. Quando attivato, se Claude 'sente' di aver bisogno di un aiuto superiore, scala temporaneamente il ragionamento a un modello più capace e poi ritorna alla modalità normale.

 **Utilità pratica:** Particolarmente utile in fase di debugging complesso o quando si affrontano problemi architetturali difficili. Aumenta leggermente il consumo di token ma può sbloccare situazioni altrimenti difficili da risolvere.

## 11. Gestione dei Token

La metodologia SDD, specialmente con i workflow avanzati, può essere token-intensive. Una pianificazione consapevole del consumo di token è fondamentale per lavorare in modo efficiente e sostenibile.

### 11.1 Le Operazioni Più Costose

- Lettura completa di tutti i documenti di un progetto (richiesta esplicita).
- Esecuzione di task in Cowork con molti documenti nella knowledge base.
- Sub-Agent Driven Development (ogni sub-agente ha il proprio contesto).
- Utilizzo del modello Opus per attività di analisi complessa.

### 11.2 Strategie per Ottimizzare il Consumo

1. Separare configurazione e sviluppo: Non fare la configurazione dell'ambiente e lo sviluppo nella stessa sessione/giornata. La configurazione è token-intensive; dedica una sessione separata.
2. Usare Sonnet per il codice: Claude Sonnet è significativamente più efficiente di Opus per le attività di sviluppo. Riserva Opus per analisi strategiche.
3. Modularizzare il CLAUDE.md: File modulari ridotti al minimo necessario consumano meno token rispetto a un file monolitico caricato intero ad ogni sessione.
4. Fare una cosa alla volta: Se si finiscono i token, chiudere la sessione e aprirne una nuova anziché cercare di forzare il completamento.
5. Pianificare gli sprint: Non iniziare uno sprint aspettandosi di finirlo nella stessa sessione se si è già a metà del limite settimanale.

### 11.3 Indicazioni per il Piano Gratuito

Con il piano standard (gratuito o base), le sessioni di sviluppo hanno un limite settimanale di utilizzo. Come riferimento orientativo emerso dalla discussione in sessione:

- Opus per analisi: circa 1 ora di utilizzo intensivo prima di raggiungere il limite di sessione.
- Sonnet per sviluppo codice: tra 3 e 4 ore di sviluppo attivo prima di raggiungere il limite.
- Attività miste: il limite dipende dalla complessità dei task e dalla lunghezza dei contesti gestiti.

**⚠ Consiglio:** Nei limiti del piano gratuito, è preferibile lavorare su attività brevi e mirate piuttosto che su sessioni lunghe e onnicomprensive. La qualità del risultato non è correlata alla lunghezza della sessione, ma alla qualità della specifica iniziale.

## 12. Riepilogo: Il Flusso Completo

---

La sessione ha dimostrato un flusso completo dall'idea iniziale al prototipo funzionante. Ecco la sintesi operativa:

### Fase 1 — Ideazione e Analisi (Claude Desktop, Chat normale)

1. Brainstorming sull'idea: definire le funzionalità desiderate, raccogliere suggerimenti dall'AI.
2. Analisi dello stack tecnologico: richiedere un confronto tra le opzioni disponibili tramite Cowork.
3. Selezione dello stack: scegliere la tecnologia più adatta al contesto.

### Fase 2 — Configurazione Progetto Claude (Claude Desktop, Progetto)

1. Creare il Progetto Claude con i tre artefatti (descrizione sintetica, project context, istruzioni).
2. Aggiungere alla knowledge base i documenti di analisi generati (framework analysis, SRS).

### Fase 3 — Generazione della Configurazione (Cowork dal Progetto)

1. Richiedere la generazione del CLAUDE.md ottimizzato e modularizzato.
2. Ottenere contestualmente: CLAUDE.md, file modulari, piano di sviluppo macro, mappa di progetto.
3. Copiare la working folder di Cowork nella cartella di sviluppo locale.

### Fase 4 — Sviluppo Iterativo (Claude Code)

1. Per ogni sprint/task: avviare /brainstorming → ottenere specifica → approvare → generare piano → eseguire.
2. Usare Sub-Agent Driven Development per task parallelizzabili.
3. Al termine di ogni sprint: aggiornare CLAUDE.md con /revise-claude-md.
4. Periodicamente: migliorare la configurazione con /claude-md-improver.

☑ **Conclusione chiave:** La qualità del risultato è proporzionale alla qualità della specifica. Questo vale per la specifica funzionale, per quella tecnica, e persino per la configurazione del CLAUDE.md. Investire tempo nella fase di definizione è sempre la scelta più efficiente.

## 13. Glossario dei Termini Chiave

| Termine                                  | Definizione                                                                                                                            |
|------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| <b>SDD (Spec-Driven Development)</b>     | Metodologia di sviluppo in cui la specifica scritta dall'utente guida il lavoro dell'AI. Contrapposto al 'vibe coding'.                |
| <b>CLAUDE.md</b>                         | File di configurazione principale di Claude Code. Equivale al 'sistema operativo' della cartella di sviluppo.                          |
| <b>Brainstorming &amp; Specification</b> | Workflow avanzato attivato dal comando /brainstorming. Produce specifiche di alta qualità tramite dialogo collaborativo.               |
| <b>Sub-Agent Driven Dev.</b>             | Tecnica di sviluppo in cui l'agente orchestratore delega task a sub-agenti che lavorano in parallelo.                                  |
| <b>Planning Mode</b>                     | Modalità di Claude Code in cui vengono eseguite solo operazioni di lettura e pianificazione (nessuna modifica ai file).                |
| <b>Metaprompting</b>                     | Tecnica che consiste nel chiedere all'AI di scrivere il miglior prompt per un compito, anziché eseguirlo direttamente.                 |
| <b>TCOF</b>                              | Schema per la struttura dei prompt: Task, Context, Output, Format. Base di tutti i prompt strutturati del corso.                       |
| <b>Human-in-the-Loop</b>                 | Principio secondo cui l'essere umano deve sempre rimanere al centro del ciclo di sviluppo, revisionando e approvando ogni passo.       |
| <b>Enforcement Rules</b>                 | Regole di comportamento inserite nel CLAUDE.md che ricordano a Claude di rispettare i principi cardine della programmazione.           |
| <b>Working Folder</b>                    | Cartella locale creata automaticamente da Cowork per organizzare i file di una sessione di lavoro.                                     |
| <b>MVP</b>                               | Minimum Viable Product: prototipo funzionante con le funzionalità essenziali, da validare prima di procedere con lo sviluppo completo. |
| <b>SRS</b>                               | Software Requirements Specification: documento formale che descrive i requisiti tecnici e funzionali di un sistema software.           |
| <b>Domain Expert</b>                     | Esperto del dominio applicativo (non necessariamente tecnico). Nella visione del corso, il ruolo ideale degli utenti non sviluppatori. |

## 14. Anticipazioni sulla Prossima Lezione

---

La sessione si chiuderà idealmente con la sesta lezione, che sarà un 'ritorno alla terra' dopo l'apice di complessità raggiunto in questa sessione. Gli argomenti previsti:

- **Skill di Claude Cowork:** Come costruire skill personalizzate per automatizzare attività professionali (traduzione documenti, verbali di riunione, web scraping, ecc.).
- **MCP Server:** Come collegare la piattaforma Claude a strumenti esterni già sviluppati tramite il Model Context Protocol.
- **Assistenti Personalizzati:** Come costruire agenti AI specializzati per collaborare nel lavoro quotidiano.

 **Promessa:** La prossima lezione sarà più accessibile rispetto a questa, con attività pratiche immediatamente applicabili anche da chi non ha competenze tecniche avanzate.